

Lab: 0110

Shift Registers & Counters – Verilog

From Flip-Flops to Sequencing Circuits

CSA222: Modern Computer Architecture

Q1

8-bit Register with Load Enable

PROBLEM

- ✦ Write reg8_load: inputs clk, load, d[7:0]; output q[7:0].
- ✦ On a rising edge: if load=1, capture d into q.
- ✦ If load=0, q holds its previous value (not X, not 0).
- ✦ No reset pin required for this exercise.

SOLUTION

```
module register_8bit (  
    input      clk,  
    input      ld,  
    input [7:0] d,  
    output reg [7:0] q  
);  
    always @(posedge clk) begin  
        if (ld)  
            q <= d;  
    end  
endmodule
```

Q2

8-bit Register with Synchronous Reset

PROBLEM

- ◆ Extend Q1: add a synchronous rst input.
- ◆ On a rising edge, if rst=1, q clears to 8'b0 — regardless of load.
- ◆ Priority order: rst beats load beats hold.
- ◆ rst only takes effect on a clock edge (NOT asynchronous).

SOLUTION

```
module register_8bit_rst (  
    input        clk,  
    input        rst,  
    input        ld,  
    input  [7:0] d,  
    output reg [7:0] q  
);  
    always @(posedge clk) begin  
        if (rst)  
            q <= 8'b00000000;  
        else if (ld)  
            q <= d;  
    end  
endmodule
```

Q3

SIPO Shift Register

PROBLEM

- ✦ Write `sipo_reg`: a 4-bit Serial-In, Parallel-Out shift register.
- ✦ Inputs: `clk`, `sin` (1 bit). Output: `q[3:0]` (parallel).
- ✦ Each rising edge: shift `q` left by one, load `sin` into bit 0.
- ✦ After 4 clocks, `q[3:0]` holds the last 4 bits received.

SOLUTION

```
module sipo (  
    input      clk,  
    input      rst,  
    input      d_in,  
    output reg [3:0] q  
);  
    always @(posedge clk) begin  
        if (rst)  
            q <= 4'b0000;  
        else begin  
            q[3] <= q[2];  
            q[2] <= q[1];  
            q[1] <= q[0];  
            q[0] <= d_in;  
        end  
    end  
endmodule
```

Q4

PISO Shift Register

PROBLEM

- ✦ Write `piso_reg`: a 4-bit Parallel-In, Serial-Out shift register.
- ✦ Inputs: `clk`, `load`, `d[3:0]`. Output: `sout` (1 bit).
- ✦ If `load=1` on a clock edge, capture `d` (parallel load).
- ✦ If `load=0`, shift right each edge; output the LSB on `sout`.

SOLUTION

```
module piso (  
    input        clk,  
    input        rst,  
    input        sh_ld,  
    input [3:0] d,  
    output reg [3:0] q,  
    output       d_out  
);  
    assign d_out = q[0];  
  
    always @(posedge clk) begin  
        if (rst)  
            q <= 4'b0000;  
        else if (!sh_ld)  
            q <= d;  
        else begin  
            q[0] <= q[1];  
            q[1] <= q[2];  
            q[2] <= q[3];  
            q[3] <= 1'b0;  
        end  
    end  
endmodule
```

Q5

4-bit Synchronous Up Counter

PROBLEM

- ✦ Write `up_counter4`: a 4-bit synchronous up counter.
- ✦ Inputs: `clk`, `rst`, `en`. Output: `q[3:0]`.
- ✦ On a rising edge: if `rst=1`, `q` clears to 0000.
- ✦ Else if `en=1`, `q` increments by 1; else, `q` holds.
- ✦ Must wrap 1111 -> 0000 with no special-case code.

SOLUTION

```
module up_counter4 (  
    input clk, rst, en,  
    output reg [3:0] q  
);  
    always @(posedge clk) begin  
        if (rst)  
            q <= 4'b0000;  
        else if (en)  
            q <= q + 4'b0001; // wraps  
                                // for free:  
                                // 1111+1  
                                // =0000  
        end  
    endmodule
```

Q6

4-bit Synchronous Down Counter

PROBLEM

- ✦ Write `down_counter4`: a 4-bit synchronous down counter.
- ✦ Inputs: `clk`, `rst`, `en`. Output: `q[3:0]`.
- ✦ On a rising edge: if `rst=1`, `q` loads to 1111 (its MAX, not 0).
- ✦ Else if `en=1`, `q` decrements by 1; else, `q` holds.
- ✦ Must wrap 0000 -> 1111 with no special-case code.

SOLUTION

```
module down_counter4 (  
    input clk, rst, en,  
    output reg [3:0] q  
);  
    always @(posedge clk) begin  
        if (rst)  
            q <= 4'b1111;    // reset to  
                             // MAX, so it  
                             // has room  
                             // to count  
                             // down  
        else if (en)  
            q <= q - 4'b0001; // wraps  
                             // 0000-1  
                             // =1111  
        end  
    endmodule
```

Key Takeaways

Two theory blocks, one skeleton each.

01

A register is n flip-flops sharing a clock; shift registers just wire neighbour-to-neighbour (SISO/SIPO/PISO/PIPO), and can add direction (bidirectional) or full mode-select (universal).

02

A ripple counter chains flip-flop clocks together — cheap, but the count is briefly wrong while it settles (ripple delay).

03

A synchronous counter shares one real clock; combinational logic computes each flip-flop's next input, so the whole count is valid the instant after the edge.

04

Ring and Johnson counters are shift registers with feedback — n states (Ring) or $2n$ states (Johnson), no decoding logic needed.

05

Every Verilog exercise in this lab is one of these skeletons: an if/else inside always @(posedge clk) — only the next-state expression changes.



**Thanks
for
watching!**